

Here's a tour script you can read or adapt, plus tech/API notes and ~30s voice-over per page.
(Based on your src/app routes and APIs.)

Stack & integrations (big picture)

Layer What you use

App

Next.js (App Router) + React + TypeScript — server components for data-heavy pages, client components where you need hooks (forms, map).

Auth

Supabase Auth (@supabase/ssr) — server client reads cookies; login uses signInWithPassword; /dashboard checks getUser() and redirects if logged out.

Maps / places

Google Maps Platform: Geocoding + Places Text Search + Place Details in src/app/api/places/route.ts with GOOGLE_MAPS_API_KEY. Results drive Leaflet (leaflet + react-leaflet) on the client.

Seed data

Curated orgs in src/lib/orgs.ts (e.g. Dallas nonprofits) when Places fails or as domain model.

Nonprofit verification

ProPublica Nonprofit Explorer API (src/lib/propublica.ts) on org profiles when an EIN exists.

AI (org copy)

Google Gemini (src/lib/gemini-org.ts, GEMINI_API_KEY) — optional “overview” for place-sourced listings, not the source of addresses/hours.

Events

SerpAPI + Google search with site:eventbrite.com (src/lib/eventbrite.ts, SERPAPI_API_KEY) — Eventbrite's old public search API is gone, so you discover real /e/ links via search results.

Housing

HUD data via your src/lib/hud.ts and HUD_API_KEY through GET /api/housing.

In-depth tour script (one continuous walkthrough)

[Open app — layout]

“Every page sits under a shared layout: Poppins font, AppNavbar on all routes except login, and a Supabase-aware shell. The navbar is a client component so it can highlight the active route and hide on /login.”

[Home /]

“Home is the intake funnel. We explain the three steps—need, location, connect—and the form is HomeIntake, a client component that collects city/ZIP, type of help, and language. Nothing hits an API here yet; on submit we router.push to /map with query params (zip, help, lang). That's how we keep state in the URL for sharing and refresh.”

[Map /map]

“The map page is a server component wrapping MapResults, which client-fetches /api/places when zip and help from the URL change. The route handler geocodes the location string, runs Places Text Search with service-specific keywords, then Place Details for phone, website, and hours. We map JSON into our Organization shape, score and filter with scoreOrgs in lib/orgs,

and show a sidebar list plus Leaflet markers. View profile uses orgProfilePath: EIN for seeded orgs, Google place_id for pure Places rows—fixing the old empty-EIN bug.”

[Organization profile /organizations/[orgId]]

“The profile is a server page. resolveOrgFromParam loads from seed SEED_ORGS by EIN or id, or calls Google Place Details for a place_id. If it’s a Places-only org, we optionally call Gemini for a short ‘what to know’ block. If there’s an EIN, we fetch ProPublica for nonprofit metadata. Contact blocks use your SVG icons; links go out to phone and website.”

[Events /events]

“Events search runs searchEvents → searchEventbrite in lib/eventbrite.ts, which is really SerpAPI calling Google with the user query, location, and site:eventbrite.com. We parse organic results, keep links that look like Eventbrite event URLs, and map them to cards. No live Eventbrite search API—that’s why this design exists.”

[Housing /housing]

“Housing is Fair Market Rent / income-style context for advocates. Params in the URL feed HousingAffordabilityResults, which calls /api/housing, which calls getHousingAffordability using HUD-backed logic and HUD_API_KEY. It’s planning data, not a rental listing site.”

[Login /login]

“Login is Supabase signInWithPassword from the browser client. Session cookies are handled by Supabase SSR helpers. After sign-in we go home or refresh.”

[Account /dashboard]

“Dashboard is server-only auth gate: createClient from cookies, getUser(), redirect('/login') if missing. We show email and a simple signed-in state—your Account nav item.”

~30 seconds per page (read-aloud scripts)

Roughly ~85–100 words each ≈ 30 seconds at a clear demo pace.

/ — Home (~30s)

“This is Sabeel’s front door. We tell users we match them to local nonprofits by what they need, where they are, and language. The form collects city or ZIP, service type—housing, food, legal, medical, and more—and language. When they continue, we don’t call an API on this screen; we send them to the map with those choices in the URL, so results are shareable and bookmarkable.”

/map — Map (~30s)

“Here search results become geography. The app calls our Places API route on the server: Google geocodes the area, searches for relevant venues, and enriches each pin with phone, site, and hours. We rank matches by service, language, urgency, and distance, show a scrollable list, and a Leaflet map with numbered markers. From here they can call an org or open a profile—using either a nonprofit EIN or a Google place id so every card resolves.”

/organizations/[orgId] — Org profile (~30s)

“This is the full organization view. Curated entries show our description and services; we add ProPublica data when we have an EIN—tax status, filings, links out. For map-only listings we pull Google Place Details and optionally Gemini for a cautious AI overview—always with a reminder to confirm on the ground. Contact is phone, web, hours, with icons instead of emoji for clarity.”

/events — Events (~30s)

“Events finds public Eventbrite pages indexed by Google. We use SerpAPI to run a search like your keywords plus city plus site:eventbrite.com, then turn organic links into cards. The old Eventbrite search API doesn’t exist anymore, so this is how we still surface real event URLs. Times in snippets aren’t reliable—users should open Eventbrite for the schedule.”

/housing — Housing (~30s)

“Housing is for affordability context—not apartment listings. Users set location, household size, bedrooms, and a monthly budget. We call HUD-backed data through our housing API to compare Fair Market Rent and income limits—useful for resettlement and nonprofit planning, with Dallas as a sensible default in the UI.”

/login — Login (~30s)

“Sign-in is powered by Supabase Auth: email and password on the client, session in cookies via Supabase’s SSR helpers. There’s no demo-account strip anymore—production-style login only. After a successful sign-in, the session is available to protected areas like the account page.”

/dashboard — Account (~30s)

“The Account screen is a simple gate: if you’re not logged in, the server redirects to login. If you are, we show your email and confirm the session is active. It proves Supabase is wired end-to-end—middleware or server checks can grow later into saved searches or preferences.”

Optional closing line (15s)

“Under the hood it’s Next.js, Supabase, Google Maps, ProPublica, Gemini, SerpAPI, and HUD—combined so refugees and newcomers get actionable, verifiable paths to help.”

If you want this tightened for a 2-minute or 5-minute demo, say which length and I’ll cut and reorder.